java-codebase-rag: A Graph-Native Code Intelligence Layer for Agentic Navigation of Java Microservice Codebases

Dmitriy Teriaev* Perplexity Computer†

May 2026

Abstract

Large language model agents reason poorly about codebases the moment the codebase no longer fits in their context window. The standard remedy — chunked retrieval-augmented generation over source files [9] — treats source code as prose and produces irrelevant or contradictory chunks for non-trivial questions ("who calls this method", "what HTTP routes does this microservice expose", "if I change this DTO, what breaks"). **java-codebase-rag** is a graph-native code intelligence layer that replaces chunk retrieval with deterministic graph navigation. It exposes a deliberately small Model Context Protocol (MCP) surface — five tools: `search`, `find`, `describe`, `neighbors`, `resolve` — backed by a typed property graph extracted from Java microservice source via tree-sitter [12]. The five tools collapse onto three primitive operations the agent actually performs: *locate* a starting node (`search`, `find`, `resolve`); *inspect* a node's full record (`describe`); *walk* edges in the graph (`neighbors`). We call this metaphor *a GPS for the agent*: the agent always knows where it is in the codebase, what is adjacent, and how to move. This report describes the architecture, the inspirations (GraphRAG [5], the MCP standard [1], and the broader AST-graph-RAG line), and the design principles that shaped a v2 redesign from a 9-tool surface into the current 5-tool surface plus eight operator CLI subcommands grouped as lifecycle (`init`, `increment`, `reprocess`, `erase`), introspection (`meta`, `tables`, `diagnose-ignore`), and analysis (`analyze-pr`) (Section 3.3 discusses the CLI briefly; full reference in `docs/JAVA-CODEBASE-RAG-CLI.md`). We do not include empirical evaluation: testing on real legacy codebases is in progress and the data is not yet ready to publish. The contribution of this report is conceptual: it argues for a particular shape of code-intelligence layer — minimal, typed, graph-native, agent-shaped — and shows the architecture in enough detail that a reader could rebuild it for another language ecosystem.

1. Frame: the agent needs a map, not a search engine

Most code-intelligence systems that surface results to language models follow one of two patterns. **Pattern A: chunked RAG over source files** — split source files into windowed chunks, embed, store in a vector database, retrieve the top-k for any natural-language query, and inject the chunks into the prompt. This is the default RAG [9] pattern transposed onto code, and it inherits all of RAG's failure modes: irrelevant chunks for structural queries, lost-in-the-middle effects [10] when several chunks must be reconciled, and silent omission of relevant code that didn't happen to embed near the query. **Pattern B: language-server bolted to a tool layer** — expose a wide surface of LSP-style operations (find references, go to definition, list workspace

*Senior software engineer, Sberbank. Correspondence: `doudmitry@gmail.com`

†Co-author in the agentic-drafting sense: ideation, structure, prose, and diagrams were produced jointly with the human author over an extended design conversation.

symbols) as agent tools. This works for single-method questions but produces a sprawling tool catalog — 20+ tools is common — that weak models cannot navigate reliably and that contains pairs of redundant operations.

We argue for a third pattern. The agent does not need a search engine; the agent needs *a map*. The map is a typed property graph extracted from the source. The agent's job is to navigate the map: pick a starting point, inspect what is there, walk to adjacent things. Three primitive operations are sufficient: **locate**, **inspect**, **walk**. Every realistic agent question over a codebase — "who calls this", "what HTTP routes exist", "what DTOs cross this boundary", "if I change this method, what depends on it" — decomposes into a chain of these three operations.

This is the GPS metaphor. The agent is always at some node in the graph. `search` and `find` get it to a starting node ("where am I?"). `describe` reports the full record at the current node ("what is here?"). `neighbors` returns adjacent nodes filtered by edge type and direction ("where can I go?"). The agent reasons; the system navigates. The agent does not retrieve text and reason about whether the text answers the question; it walks deterministic edges and reasons about the structure.

Reframing code intelligence as graph navigation has three immediate consequences. First, the tool surface shrinks dramatically and stops growing with new use cases — new questions become new *chains* of the existing tools, not new tools. Second, the system never returns "approximately relevant" results; every walk is exact (the edge either exists in the graph or it does not). Third, the failure mode shifts from "the model hallucinated" to "the graph is missing an edge type", which is a tractable engineering problem rather than a probabilistic one.

2. Inspirations

This system is not a research artefact. The architecture stitches together ideas from four streams of recent work; this section credits each.

GraphRAG and AST-graph retrieval. Microsoft's GraphRAG [5] demonstrated that constructing a knowledge graph from source documents and retrieving over the graph — with community detection for global summarisation and hierarchical context for local queries — substantially outperforms flat vector retrieval for query-focused summarisation. The lesson generalises beyond text: *any* corpus with strong relational structure benefits from extracting that structure once and querying it as a graph. Source code has the strongest relational structure of any corpus we routinely retrieve over (compilers extract it for free); not exploiting it is a missed opportunity. Subsequent work in the AST-graph-RAG line [6] reinforces the pattern: lighter, simpler graph backends with cleaner ontologies often beat heavyweight ones for retrieval quality.

Context engineering. Context engineering — filling the context window with the right information at the right time — shaped our tool design. A tool that returns 5 KB of plausibly-relevant text is not equivalent to a tool that returns 200 bytes of structured truth; the second is far more useful. Lost-in-the-middle effects [10] mean that long noisy contexts actively hurt downstream reasoning. The correct response is not "retrieve better" but "retrieve less, with type guarantees". Every tool in the java-codebase-rag surface returns small, typed, deterministic JSON. The agent never has to decide whether what it received is signal or noise.

Model Context Protocol. The MCP standard [1] fixed the impedance mismatch between agents and tools: a single transport, a single way to declare schemas, and a single way to bind tools to hosts (Claude Code, Cursor, Qwen Code, and others). Without MCP this report would describe a Claude-Code-only system. With MCP, a single Python server reaches every host the user already prefers. The standard does one thing well and stops.

The agent-skills layer. Anthropic's agent skills [2] provided the missing piece between raw tool calls and agent reasoning: a skill is a slash-invokable, declaratively-described chain of tool calls that encodes a recurring intent ("trace this request flow", "show me callers of this method"). Skills are how a small fixed MCP surface grows into hundreds of usable agent intents without growing the tool count. We describe the planned skills layer briefly in §7 and defer its specification to a separate document; empirical testing showed that a comprehensive prose guide (mirrored as `docs/AGENT-GUIDE.md`) is sufficient for current weak-model performance, so the skills layer is not yet on the critical path.

What we are not. We do not claim novelty over GraphRAG, LightRAG, or LSP-backed tooling. We claim that a particular synthesis — minimal MCP surface, typed property graph, three-primitive navigation model, agent-shaped affordances — is the right shape for code intelligence at the agentic-development layer. The synthesis is the contribution.

3. Architecture: three layers

The system is organised in three layers (Figure 1). Each layer has a single concern; layer boundaries are crossed only by typed contracts.

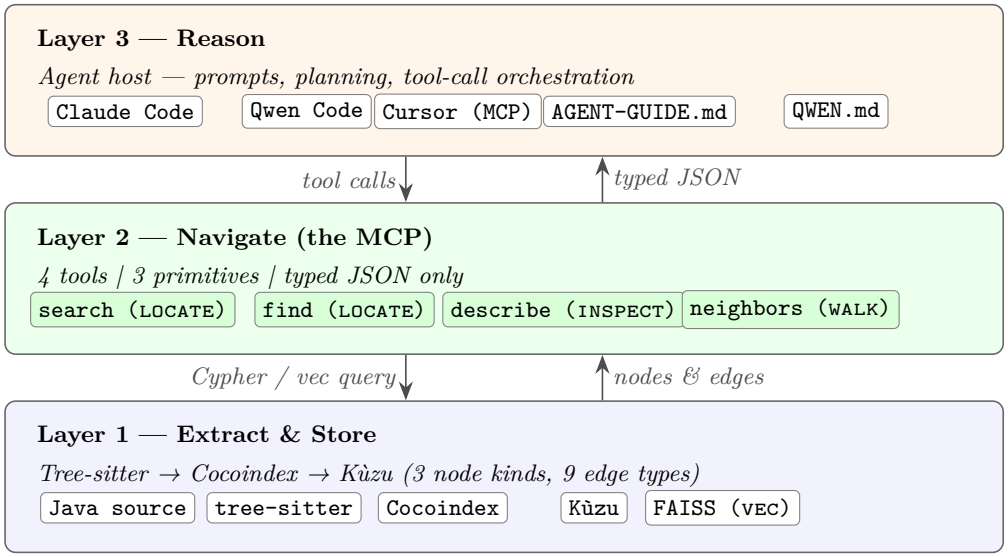


Figure 1: Three-layer architecture. **Layer 1 (Extract & Store)** ingests Java source via tree-sitter and emits nodes and edges into an embedded property graph. **Layer 2 (Navigate)** exposes five MCP tools collapsing onto three primitive operations: locate, inspect, walk. **Layer 3 (Reason)** is the agent host (Claude Code, Qwen Code, or any MCP-compatible runtime) plus optional skills. Each layer is replaceable: swap the extractor for another language, swap the graph backend, swap the agent host — contracts hold.

3.1 Layer 1: extract and store

The extractor walks a Java source tree using tree-sitter [12] grammars. Tree-sitter gives an incremental, error-tolerant parse: the system handles syntactically-broken legacy code without aborting, which matters for the target environment (large Java microservice estates with ageing modules). The parser visits each declaration and emits typed graph nodes; a second pass resolves cross-file references and emits edges.

The graph is stored in Kùzu [8], an embedded property-graph database with a Cypher-style query language. Kùzu was chosen over Neo4j for two pragmatic reasons: it embeds in-process (no separate database server to operate), and it has zero ongoing cost for a developer-side tool. Cocoindex [3] drives the indexing pipeline (incremental rebuilds, dependency tracking, change detection on source files).

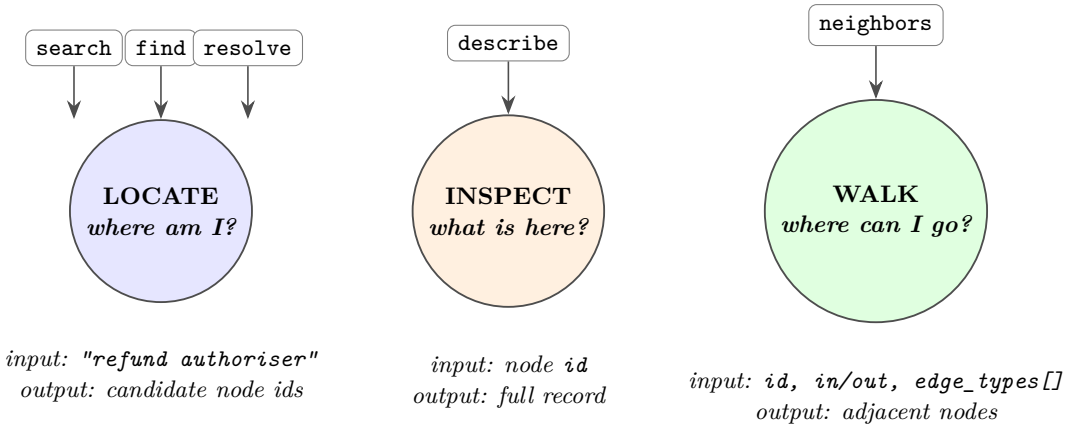
The ontology is small by design (Table 1). Three node kinds (SYMBOL, ROUTE, CLIENT) cover everything an agent realistically asks about; ten edge types cover every relation we have found load-bearing in practice. The ontology version is currently 13; the discipline is that no edge type ships unless at least three realistic agent questions in our use-case backlog require it.

Table 1: The current ontology. Node kinds and edge types are deliberately few. The constraint is that every type pays for itself by enabling at least three concrete agent questions.

Element	Description
Node kinds	
SYMBOL	A Java declaration: class, interface, enum, record, annotation, method, or constructor. The dominant node kind.
ROUTE	An HTTP-exposed endpoint (Spring `@RequestMapping`, JAX-RS `@Path`, etc.). One per HTTP method + path.
CLIENT	A typed reference to an external service: declared HTTP client (Feign, OpenFeign, REST clients) or messaging client.
Edge types (10)	
EXTENDS	Class- or interface-inheritance edge (Java `extends`).
IMPLEMENTS	Interface-implementation edge (Java `implements`).
INJECTS	Dependency-injection edge: a symbol receives another via constructor, field, or setter injection.
OVERRIDES	Subtype method overrides a supertype declaration with the same signature.
DECLARES	Container declares a member: class DECLARES method, package DECLARES class, etc.
DECLARES_CLIENT	Module declares a typed external-service client.
CALLS	In-process method call. The work-horse edge.
EXPOSES	A controller method EXPOSES an HTTP route.
HTTP_CALLS	Method invokes a declared HTTP client (cross-service edge).
ASYNC_CALLS	Method publishes to / consumes from a message broker (Kafka, RabbitMQ).

3.2 Layer 2: navigate (the MCP)

Layer 2 is the public surface of the system. Five tools, three primitive operations (Figure 2). Every tool returns small, typed JSON; no tool returns more than a single page of results without explicit pagination. Successful outputs may also carry advisory `hints` (templated next-call strings) and pagination echoes on `search` / `find`.



Every agent question = a chain over these three primitives.

Figure 2: The GPS metaphor. Five MCP tools collapse onto three primitive operations: *locate* (where am I?), *inspect* (what is here?), *walk* (where can I go?). Every realistic agent question decomposes into a chain over these three primitives. The agent supplies the reasoning; the graph supplies the geometry.

`search` — locate by meaning. Semantic vector search over symbol descriptions, route paths, and client identifiers. Embeddings are produced by a sentence-BERT model [11]; the index is FAISS-backed [7]. Returns ranked candidates with their `id`, `kind`, and a one-line summary. The agent uses `search` when it has a fuzzy starting query in natural language ("where do we authorise refund requests").

`find` — locate by structure. Filtered structured query over the graph. Takes a `kind` (`symbol`, `route`, or `client`) and a `NodeFilter` (15 optional fields: 3 universal, 5 symbol-specific, 3 route-specific, 4 client-specific). Returns deterministic, exhaustive results — no ranking, no embeddings. The agent uses `find` when it knows the shape of what it wants ("all controller methods in the `auth` module exposed at `POST /authorise`").

`describe` — inspect. Returns the complete record for a single node by `id`, including all properties, the source file and line range, and a one-line summary. This is the only tool that returns more than identifiers and edge labels — it is the agent's "zoom in" operation.

`neighbors` — walk. Graph traversal. Takes a node `id`, a `direction` (`in` or `out`), and a non-empty list of `edge_types`. Returns adjacent nodes one hop away, optionally filtered by a `NodeFilter` on the destination. Multi-hop walks are explicit chains of `neighbors` calls — the system never silently follows multiple hops, because that breaks the agent's ability to reason about what it just did.

The full type signatures are listed in Appendix A. Two design choices in the current shape are worth surfacing:

1. **`NodeFilter` is one shared type, not three.** It has fields specific to symbol/route/client kinds, and the server validates that the filter is applicable to the queried kind. The alternative — three separate filter types — triples the schema surface for weak models with no expressive gain.
2. **Tool parameters are `Literal`-typed where finite.** `kind`, `direction`, `edge_types`, and `symbol_kind` all carry literal-string types in the schema, which gives weak models a closed

enumeration to choose from. This was a v2.x improvement after observing weak models invent values like `kind="function"` in production traces.

3.3 Layer 3: reason (the agent)

Layer 3 is whatever MCP-compatible host the developer prefers — Claude Code, Qwen Code, Cursor, or another runtime. The host loads the java-codebase-rag MCP server, sees the five tools, and the agent reasons over them. There is no logic in this layer that is specific to java-codebase-rag; the entire affordance is the five tools and a prose agent guide (`docs/AGENT-GUIDE.md`) that documents the canonical workflows — forced reasoning preamble, decision tree, edge taxonomy, worked examples.

A planned addition (deferred) is a thin skills layer that turns recurring intents (`/callers`, `/routes`, `/explain-feature`) into one-line slash invocations that compile to MCP-call chains. Empirical testing on the target codebase showed that the prose guide alone is sufficient for current weak-model accuracy, so the skills layer is not yet implemented.

4. Agent workflow

Figure 3 traces a representative interaction. The user asks "who calls `RefundService.approve`". The agent decomposes the question into three calls — `search` to locate the candidate symbol, `describe` to confirm the right one, `neighbors(in, [CALLS])` to walk inbound call edges — and reports the answer. The agent does not retrieve source files. The agent does not reason about whether retrieved chunks are relevant. The agent navigates.

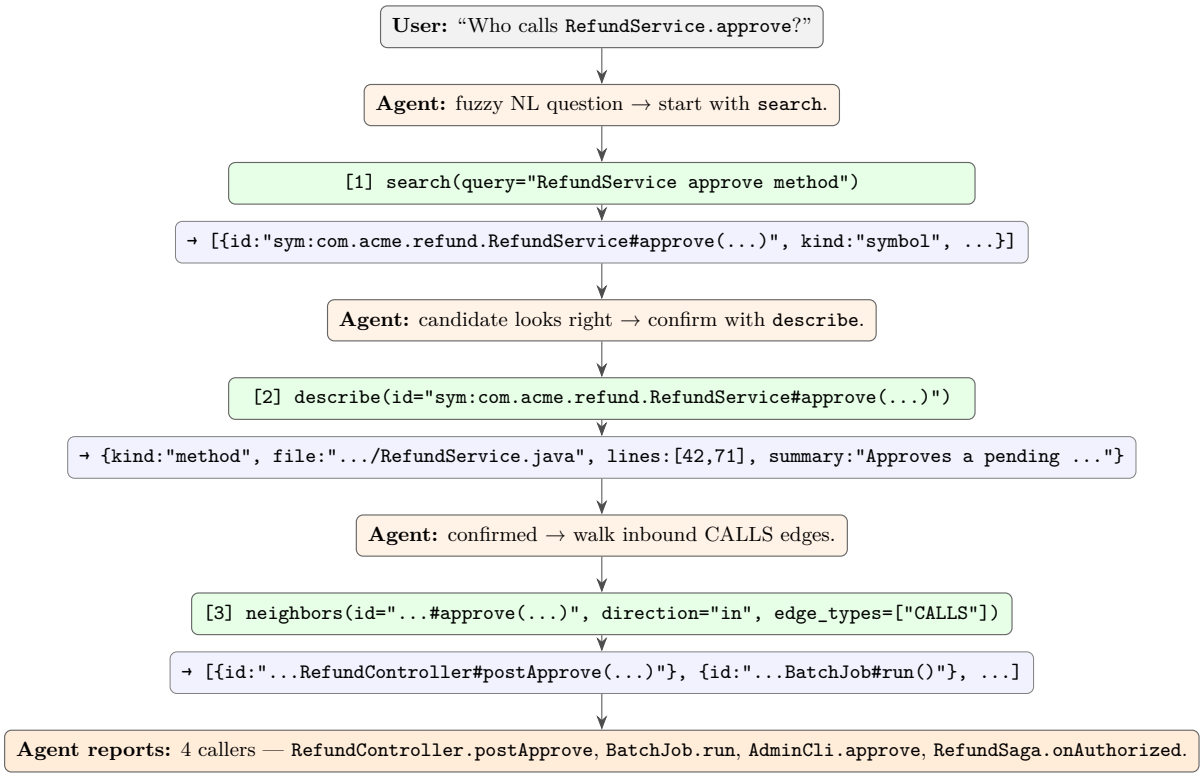


Figure 3: A canonical agent interaction. Three tool calls, deterministic edges, typed JSON. Every step is auditable: the user can see exactly which graph edges were walked. There is no chunk retrieval; there is no scoring threshold to tune.

The workflow is general. The java-codebase-rag agent guide enumerates 15+ canonical intents,

each decomposing into 2–5 tool calls over the same five tools. New intents are new chains, not new tools.

A subtle point worth surfacing: the agent's first move is almost always `search`, not `find`. Natural-language questions arrive fuzzy; `search` is the cheapest way to get from "fuzzy English" to "candidate node ids". Once the agent has an id, all subsequent navigation is deterministic. This shape — semantic at the boundary, structural in the interior — is the practical reconciliation between vector retrieval and graph navigation. We do not need hybrid retrieval (parallel vector + graph fusion) for most questions; we need *sequential* retrieval, with the vector step strictly outside the graph step.

5. Design principles

The architecture above did not arrive in one step. It arrived after a v1 surface of nine MCP tools collapsed — in design review — into a small fixed set (currently five navigation tools). Five design principles guided the collapse, and they remain binding for any future surface change.

1. **The MCP is a graph navigator, not a reasoning engine.** If a tool's job is to combine results, summarise across nodes, or rank candidates, the tool belongs in the agent's reasoning, not the MCP. Move it to a skill or to a CLI subcommand.
2. **Every tool has a single primitive operation.** If a tool does two things ("search and describe", "find and walk"), it should be two tools or one tool with a strict contract. Composite tools confuse weak models and double the failure surface.
3. **Schemas are closed where finite.** Every parameter that takes one of N known values is a `Literal`-typed enumeration. The agent gets explicit affordances; the server gets free input validation.
4. **Operational surface is human-shaped, not agent-shaped.** Tools the agent does not call belong in a CLI (`java-codebase-rag` lifecycle and introspection subcommands — `init`, `increment`, `reprocess`, `erase`, `meta`, `tables`, `diagnose-ignore`, `analyze-pr`), not the MCP. The MCP navigation surface stays small and fixed; new agent intents are new chains over the existing tools, not new tools.
5. **Walks are explicit, not silent.** The system never multi-hops without the agent asking. Multi-hop intents become multi-call chains. The agent always knows what edges it walked, and the user always can audit.

The combined effect of these principles is that the MCP surface is provably small and grows slowly. It is allowed to add edge types (with the three-question rule), and it is allowed to refine schemas; it is not allowed to grow the tool count.

6. What this deliberately does *not* do

A small system advertises its boundary. java-codebase-rag does not, and will not without a strong forcing function:

- **Generate code.** The agent's host already does this. java-codebase-rag is the navigation layer feeding the code-generating agent context; conflating roles destroys both.
- **Track version-control history.** `git blame`, `who-changed-this`, ownership inference — all out of scope until VCS data is modelled, which it currently is not.
- **Cross language boundaries.** Java only. Polyglot support is a multi-PR effort that the current target environment does not need.

- **Run hybrid retrieval** (parallel vector × graph fusion via reciprocal rank fusion or similar). The case for sequential retrieval (semantic at the boundary, structural inside) is strong; hybrid is research-only on the backlog.
- **Summarise.** If the agent wants a summary, it asks the host model to summarise the deterministic JSON it just retrieved. java-codebase-rag never produces prose.

7. Future work

Three threads are open and prioritised. **(1) Real-codebase evaluation.** Testing on a large legacy Java microservice estate is in progress; once stable, we expect to publish accuracy numbers (intent → correct-tool-chain rate, end-to-end answer correctness against human labels) for the five-tool surface against weak (Qwen) and strong (Claude Sonnet 4.5) hosts. **(2) Skills layer.** A 13-skill set — 10 single-call navigation skills (`/callers`, `/callees`, `/routes`, `/controllers`, ...) and 3 multi-step workflow skills (`/explain-feature`, `/impact-of`, `/trace-request-flow`) — is designed and on hold until the prose-guide approach shows insufficient. **(3) Tier-2 incremental rebuilds.** Today the index rebuilds the affected modules; we want commit-level incremental rebuilds for sub-second index updates on large monorepos.

We deliberately list *no* item that would grow the MCP tool count without proof that the existing five tools cannot accommodate a real intent.

Acknowledgements

This system was designed and built collaboratively over a long-running design conversation between the human author and Computer (Perplexity). Computer contributed the v2 collapse argument, the GPS framing, the schema-flattening fix for weak-model JSON encoding, and substantial portions of the prose in this report. The human author retained final design authority on every locked decision and ran every test.

References

[1] Anthropic. Model context protocol specification. Online specification, 2024. URL https://modelcontextprotocol.io. Accessed 2026-05-08.

[2] Anthropic. Agent skills. Anthropic engineering documentation, 2025. URL https://docs.anthropic.com/en/docs/build-with-claude/skills. Accessed 2026-05-08.

[3] Cocoindex contributors. Cocoindex: open-source etl framework for real-time AI data. Online documentation, 2024. URL https://cocoindex.io. Accessed 2026-05-08.

[4] Cognition AI. Don't build multi-agents. Cognition engineering blog, 2025. URL https://cognition.ai/blog/dont-build-multi-agents. Accessed 2026-05-08.

[5] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization, 2024. URL https://arxiv.org/abs/2404.16130.

[6] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. LightRAG: Simple and fast retrieval-augmented generation, 2024. URL https://arxiv.org/abs/2410.05779.

[7] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. In *IEEE Transactions on Big Data*, 2019. URL https://arxiv.org/abs/1702.08734.

[8] Kùzu contributors. Kùzu: An embedded graph database for query speed and scalability. Online documentation, 2023. URL https://kuzudb.com. Accessed 2026-05-08.

[9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks, 2020. URL https://arxiv.org/abs/2005.11401.

[10] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts, 2023. URL https://arxiv.org/abs/2307.03172.

[11] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks, 2019. URL https://arxiv.org/abs/1908.10084.

[12] Tree-sitter contributors. Tree-sitter: An incremental parsing library. Online documentation, 2018. URL https://tree-sitter.github.io/tree-sitter/. Accessed 2026-05-08.

A. Tool signatures (verbatim)

The five MCP tools have the following Python (Pydantic) signatures, simplified for readability. The full source lives at `mcp_v2.py` in the repository.

```
DeclarationSymbolKind = Literal[
    "class", "interface", "enum", "record",
    "annotation", "method", "constructor"
]

EdgeType = Literal[
    "EXTENDS", "IMPLEMENTS", "INJECTS", "OVERRIDES", "DECLARES",
    "DECLARES_CLIENT", "CALLS", "EXPOSES",
    "HTTP_CALLS", "ASYNC_CALLS"
]

class NodeFilter(BaseModel):
    # Universal
    microservice: str | None = None
    module: str | None = None
    name_contains: str | None = None
    # Symbol-specific
    symbol_kind: DeclarationSymbolKind | None = None
    symbol_kinds: list[DeclarationSymbolKind] | None = None
    fqn_contains: str | None = None
    role: str | None = None
    annotated_with: str | None = None
    # Route-specific
    http_method: str | None = None
    path_contains: str | None = None
    # Client-specific
    client_kind: str | None = None
    target_service: str | None = None

def search(query: str, limit: int = 25) -> list[NodeRef]: ...

def find(
    kind: Literal["symbol", "route", "client"],
    filter: NodeFilter | dict | str,
    limit: int = 25,
```

```
    offset: int = 0,
) -> FindOutput: ...

def describe(id: str) -> DescribeOutput: ...

def resolve(identifier: str, hint_kind: str | None = None) -> ResolveOutput: ...

def neighbors(
    id: str,
    direction: Literal["in", "out"],
    edge_types: list[EdgeType],  # min length 1
    filter: NodeFilter | dict | str | None = None,
    limit: int = 50,
) -> NeighborsOutput: ...
```

The CLI subcommands (`init`, `increment`, `reprocess`, `erase`, `meta`, `tables`, `diagnose-ignore`, `analyze-pr`) are documented separately in `docs/JAVA-CODEBASE-RAG-CLI.md` and are not part of the MCP surface.

Multi-agent considerations and host-collision behaviour are discussed in the linked design conversation; see also [4].